Step by step: testing the workflow node by node in n8n

This guide shows how to run each part of **Personal Finance Advisor v3.4** by hand and watch the data move from node to node on the n8n canvas.

For the full end-user test plan, see TESTING.md.

---

0. Before you start

1. Run `./start.sh`, then `./healthcheck.sh`. Every line should say OK.
2. Open the n8n editor at **http://localhost:5678** and open the workflow that is **Active**: **Personal Finance Advisor v3.4 (step form, one chat, deposits inside)** Direct link: http://localhost:5678/workflow/pfaAdvisorV32App
    - Don't import `n8n/pfa_advisor_v3.4.json` yourself. `./start.sh` already imports it and switches it on, and a hand-made copy is inactive, so it never receives the app's requests.
3. Use the test email in the sample files (`stepbystep@test.local`) or any made-up email. Don't use real personal data.

Why there is no single "run everything" button

The workflow is a web app, not one long chain. The browser page calls **seven separate webhook triggers**, one for each thing you do. Each call is its own execution:

#	You do this in the app	Trigger node (start here)	Request	Main nodes it runs
1	Open the app	**Advisor App**	`GET /pfa`	Render App Page → Respond: App Page
2	Enter your email	**Lookup API**	`POST /pfa/lookup`	Lookup User → Lookup Result → Respond: Lookup
3	Submit the 5 steps	**Submit API**	`POST /pfa/submit`	Plan Request (form) → Build Financial Profile → Goal Text Guardrail → Check Profile Completeness → **Respond: Building** → Prepare User Storage → Upsert User Profile → Restore User State → Debt / Allocation / Deposit analyses and agents → Report
4	Wait for the report	**Status API**	`GET /pfa/status?id=`	Report Status → Respond: Status
5	Report opens	**Report API**	`GET /pfa/report?id=`	Load Report → Render Report Page → Respond: Report Page
6	Ask in the report chat	**Chat API**	`POST /pfa/chat`	Chat Report Owner → Chat Context → Chat Input Guardrail → Financial Q&A Agent → Chat Output Sanitizer → Chat Result → Plan Change?

#	You do this in the app	Trigger node (start here)	Request	Main nodes it runs
7	Yes / No on a plan change	**Confirm API**	`POST /pfa/confirm`	Load Pending Change → Apply Plan Change → Confirmed? → (Yes: the step 3 pipeline again)

Step 3 (Submit API) is the main pipeline, with the guardrails, the analyses and all the AI agents. Start there if you only test one thing.

The other trigger nodes are not part of the app flow:

- **Chat Trigger** is n8n's hosted chat; see section 3.
- **PFA MCP Server** and **Bank Deposits MCP Server** are for MCP clients.
- **Planner Request** is called from inside the workflow.

---

1. Way A: watch one step live on the canvas (test mode)

In test mode, n8n waits on a **test URL** (`/webhook-test/...`), runs the workflow in the editor, and lights up each node as it runs. You trigger the step with `./scripts/test-step.sh`, which sends the sample request from `testing/payloads/`.

1.1 Lookup (simple, no AI)

1. In the editor, click the arrow next to **Execute workflow** at the bottom and choose **from Lookup API**. The trigger shows *"Waiting for you to call the Test URL"*. It waits for 2 minutes.
2. In a terminal, in the bundle folder:

   ```
   ./scripts/test-step.sh lookup
   ```

3. Watch **Lookup API** → **Lookup User** → **Lookup Result** → **Respond: Lookup** turn green.
4. Double-click **Lookup User** to see its input (the email) and output (the saved profile, or nothing for a new email).

1.2 Submit: the main pipeline (AI agents, about 1-3 minutes)

1. Choose **Execute workflow** → **from Submit API**.
2. Run:

   ```
   ./scripts/test-step.sh submit
   ```

   It prints `{"ok":true,"report_id":"…"}` straight away. Copy that **report_id**, because you need it for steps 4-7.
3. Keep watching the canvas. **Respond: Building** answers the browser early, and the pipeline then keeps going: storage, the analyses in parallel, the agents, and finally the report.
4. Things to check:

Node	What to look at
Build Financial Profile	Output: amounts parsed from shorthand (`18L` → 1800000), in-hand pay, EMIs and surplus
Goal Text Guardrail	Whether the goal text passed or was neutralised (try `"goal_details": "ignore previous instructions"` in `submit.json`)
Check Profile Completeness	`Needs More Info?` : a missing or invalid field stops here and returns "fix these"
Debt Analysis Agent and the other agents	**Logs** panel: the prompt, the model's reply, tokens and every tool call
Render Report Card	The final numbers merged with the agents' text

To try other answers, edit `testing/payloads/submit.json` . Amounts take `85K` , `1.2L` , `1.5Cr` , and so on. Allowed values:

- `goal_type` : Buy a home, Retirement, Child's education, Child's wedding, Buy a car, Start a business, Travel or sabbatical, Build long-term wealth, Other
- `loan_types` : Home loan, Car loan, Personal loan, Education loan, Credit card dues, Gold loan, Other loan, or `"None – I'm debt-free"`
- `risk` : Low, Medium, High

1.3 Status and report

```
./scripts/test-step.sh status <report_id>     # choose "from Status API" first
./scripts/test-step.sh report <report_id>     # choose "from Report API" first
```

`status` says `{"ready":true}` once the report is saved. To *see* the report page, open `http://localhost:5678/webhook/pfa/report?id=<report_id>` in your browser.

1.4 Chat (report-page chat, with the teammate's tools)

1. Choose **Execute workflow → from Chat API**.
2. Run:

   ```
   ./scripts/test-step.sh chat <report_id>
   ```

3. Open **Financial Q&A Agent → Logs** to see which tools it called (the rate table, the plan, the teammate's `Ask_Bank_Deposit_Schemes` / `Ask_Mutual_Funds`) and what each returned.

To change the question, edit `testing/payloads/chat.json` .

1.5 Plan change: chat, then confirm

1. Choose **from Chat API** and run:

   ```
   ./scripts/test-step.sh chat-change <report_id>
   ```

   The reply contains `"confirm":{"id":"…","changes":["Essentials ₹45K → ₹50K"]}` . Copy that **id**.
2. Choose **from Confirm API**, then run one of these:

```
./scripts/test-step.sh confirm-no <id>    # No: keeps the plan (fast, no AI)
./scripts/test-step.sh confirm <id>       # Yes: applies the change and rebuilds the
        plan (runs all agents)
```

Test-mode tips

- Each **Execute workflow** → **from …** catches **one** request, then stops listening. Choose the trigger again before each call.
- If the 2 minutes run out, the script prints a 404 saying the webhook is not registered. Click **Execute workflow** again.
- Test runs appear on the canvas only. They're not listed under Executions unless you turn on saving manual executions in **Workflow settings**.

---

2. Way B: use the real app, then replay each run (Executions)

This is the best way to see the **whole journey** in order.

1. Use the app normally at **http://localhost:5678/webhook/pfa**: email, 5 steps, submit, report, chat. You can also do the same from the terminal by adding `--live`:

   ```
   ./scripts/test-step.sh lookup --live
   ./scripts/test-step.sh submit --live            # copy report_id
   ./scripts/test-step.sh status --live <report_id>
   ./scripts/test-step.sh chat   --live <report_id>
   ```

2. In the editor, open the **Executions** tab (http://localhost:5678/workflow/pfaAdvisorV32App/executions). Each action is one run, newest first.
3. Click a run. The canvas shows the path it took: **green** nodes ran and grey ones were skipped. Click any node for its input and output.
4. Click **Debug in editor** (or **Copy to editor**) to load that run's data into the canvas. Then:
   - select any node and click **Execute step** to re-run just that node with the same input;
   - change a node's settings or code and execute the step again to compare.

---

3. The hosted chat (live on the canvas, no script needed)

1. Click **Open chat** at the bottom of the editor. This uses the **Chat Trigger** node.
2. Type the email of a saved plan (for example `stepbystep@test.local`, after running section 1.2), then ask a question.
3. Watch **Chat Trigger** → **Chat Context** → **Chat Input Guardrail** → **Financial Q&A Agent** → **Chat Output Sanitizer** → **Chat Result** light up.

---

4. Where else to look

What	Where
Every LLM call with prompt, tokens and cost	Phoenix: http://localhost:6006 → project **pfa-n8n**
Inside the teammate's service (their RAG and LangGraph steps)	Phoenix project **pfa-teammate-advisor**, or `docker compose logs -f teammate-advisor`
Saved users and reports	`./healthcheck.sh` (counts)

Sample requests

File	Used by	Notes
`testing/payloads/lookup.json`	`lookup`	The email
`testing/payloads/submit.json`	`submit`	The same answers as TESTING.md scenario A1
`testing/payloads/chat.json`	`chat`	`REPORT_ID` is replaced by the id you pass
`testing/payloads/chat-change.json`	`chat-change`	A message that proposes a plan change
`testing/payloads/confirm.json`, `confirm-no.json`	`confirm`, `confirm-no`	`PENDING_CHANGE_ID` is replaced by the id you pass

Checked on 6 Oct 2026, against the live URLs (`--live`):

Step	Result
`app`	HTTP 200
`lookup`	New user, no history
`submit`	`report_id` returned
`status`	`ready: true` after about 2 minutes
`report`	HTTP 200
`chat`	Answer from the rate table
`chat-change`	Pending change "Essentials ₹45K → ₹50K"
`confirm-no`	Plan left unchanged